App Development Project Report

Trekking Management Portal

1. Student Details

Name: Navjot Singh Dadiala

Roll Number: 23F2000835

Email: 23f2000835@ds.study.iitm.ac.in

About Me: I am a professional level student with interest in web development, python and backend application development.

---

2. Project Details

Project Title: trekking-management-application

Problem Statement:

To make and build a web-based trekking management portal that helps an organization manage hiking trails, schedule treks, assign guides, handle registrations, and track booking records. The system lets guides sign up and request admin approval. Once approved, guides can view their assigned treks, modify available slots, update trek statuses (Open/Closed/Ongoing/Completed), and manage the participant list. Trekkers can register, log in, browse open treks, view descriptions, buy tickets, track status, and view past completed/cancelled trekking history.

Approach:

The application is built using Flask as the backend with a clean blueprint-based modular structure (auth, admin, staff, user). SQLite is utilized as the database, generated dynamically through SQLAlchemy inside the app context. The frontend is built using standard Jinja2 templates, raw HTML, and clean, basic Bootstrap classes with minimal CSS to match requirements. Programmatic seeding is provided via seed.py. Easy-to-explain inline auth checks are implemented in view endpoints instead of complex decorators.

---

3. AI/LLM Declaration

I used AI/LLM assistance to help write clean templates, structure the SQLite database schemas, and create helper files. The overall extent of AI/LLM usage is approximately 15–20%, primarily limited to boilerplate routing configuration, basic table alignment, and verification report documentation formatting. All core logic (such as slot availability subtraction, booking prevention on closed/full treks, guide assignment checks, and cascade deletions) was verified and tested manually.

4. Technologies and Frameworks Used

Technology / Library	Purpose
Flask	Core backend web framework
Flask-SQLAlchemy	ORM for handling database models and queries
SQLite	Lightweight local database for storing application data
Jinja2	Template engine for rendering dynamic HTML pages
Bootstrap 5	Frontend layout framework for basic form structures
Python	Programming language used for server-side logic

---

5. Database Schema / ER Diagram

Tables:

- **User** — stores basic account details for all users (id, username, email, full_name, password_hash, role: admin/staff/user, is_approved, is_blacklisted)
- **StaffProfile** — stores staff-specific guide information linked to a user account (id, user_id, name, contact_details, experience_years)
- **Trek** — stores hiking event schedules (id, name, location, difficulty: Easy/Moderate/Hard, duration, start_date, end_date, max_slots, available_slots, price, description, status: Pending/Approved/Open/Closed/Completed, assigned_staff_id)
- **Booking** — stores trekker registrations (id, user_id, trek_id, num_tickets, total_price, status: Booked/Cancelled/Completed, booking_date, payment_status)

Relationships:

- One-to-One: User → StaffProfile
- One-to-Many: User → Booking (a trekker has many bookings)
- One-to-Many: Trek → Booking (a trek event has many bookings)
- One-to-Many: User (Staff) → Trek (a staff member is assigned to many treks)

6. API Resource Endpoints

Endpoint	Method	Description
/register	GET, POST	Register new user or staff guide
/login	GET, POST	Authenticate credentials and redirect to dashboard
/logout	GET	End active session and redirect to homepage
/admin/dashboard	GET	Display stats, treks list, approved/pending guides, and booking logs
/admin/trek/create	GET, POST	Create a new trekking route and schedule
/admin/trek/edit/<id>	GET, POST	Edit parameters of an existing trek
/admin/user/delete/<id>	POST	Completely delete a user or guide account from system
/staff/dashboard	GET	Guide dashboard showing assigned trips and trekker counts
/staff/trek/manage/<id>	GET	View list of hikers registered for a guide's assigned trek
/staff/trek/update-slots/<id>	POST	Update available slot remainder count
/staff/trek/update-status/<id>	POST	Update trek status (Open, Closed, Ongoing, Completed)
/user/dashboard	GET	Browse open treks and display quick active booking overview
/user/trek/<id>	GET	View description, difficulty details, and reservation form
/user/book/<id>	POST	Book tickets. Checks limits, status, and duplicate bookings
/user/my-bookings	GET	Display active bookings with cancel trigger option
/user/history	GET	Display records of completed and cancelled bookings history

7. Architecture and Features

Architecture Overview:

- **app.py** – Main Flask application entry point, context setup, and blueprint registration
- **extensions.py** – Declares shared database and login-manager objects independently to avoid import loops
- **models.py** – Defines relational SQLAlchemy schema structures for User, StaffProfile, Trek, and Booking
- **seed.py** – Executable command utility to build and pre-seed mock treks and test accounts
- **routes/** – Directory hosting distinct blueprints for auth, admin, guide staff, and trekker user controllers
- **templates/** – Contains clean, minimalistic Jinja2 templates matching the zero-CSS viva preferences

Implemented Features:

- User registration and login validation
- Role-based route control blocks utilizing flask-login properties
- Interactive guide sign-up approvals/deletion dashboard
- Trail slots overbooking validation and duplicate booking prevention checks
- Soft cancellations that free slots, and separate past completed history grids
- Account blacklist / deactivation flags that immediately lock user logins
- Database cascade structures to prevent orphan rows upon user deletion

---

8. Video Presentation

Drive Link:

DRIVE LINK

Or

https://drive.google.com/file/d/1N_1CalcP_latkJRtgfSh4GUAxbuxxbWC/view?usp=sharing